

Integrated Mini-SOC Ecosystem

Digital Egypt Pioneers Initiative (Round - 4)

Group : CAI4_ISS6_G1

Track: Infrastructure & Security (Information Security Analyst)

Team Members

Adham Hany Mohamed

Ahmed Mokhtar Helmy

Ibrahim Hussien Ibrahim

Zeinab Ahmed Abdelhamid

1. Project Overview & Objectives

The Mini SOC (Security Operations Center) system is a lightweight, yet fully functional security monitoring environment designed to collect, process, correlate, analyze, and visualize security events from multiple heterogeneous data sources in real-time. It simulates a production-grade SOC infrastructure capable of detecting threats, triaging incidents, generating actionable alerts, and supporting forensic investigation workflows using centralized logging and SIEM capabilities. This project was developed as the capstone deliverable of the Digital Egypt Pioneers Initiative (DEPI) — Round 4, by a team of four SOC analysts. It bridges the gap between theoretical cybersecurity training and hands-on, real-world defensive operations by building an end-to-end detection and response pipeline from scratch.

1.1 Background & Motivation

Modern organizations face an increasingly sophisticated threat landscape where cyberattacks have evolved from simple opportunistic intrusions to complex, multi-stage operations conducted by organized threat actors. The volume and velocity of security events generated by enterprise environments have grown to a scale that makes manual monitoring not only impractical but operationally impossible. A Security Operations Center (SOC) provides the centralized capability to continuously monitor, detect, analyze, and respond to cybersecurity threats. This project replicates a scaled-down but architecturally authentic SOC environment using open-source and enterprise-grade tools, enabling the team to develop practical competencies in blue team operations, threat detection engineering, and incident response.

1.2 Project Scope

The scope of this project encompasses the full lifecycle of a SOC deployment, from infrastructure provisioning and agent installation to alert creation, incident triage, and documentation. The environment integrates cloud and on-premises components to reflect realistic enterprise network topologies.

The project covers the following core domains:

- **Infrastructure Deployment:** Cloud VPS provisioning on Contabo-on-premises Windows Server 2022 Domain Controller, Windows 11 Pro workstation, and Ubuntu desktop endpoints.

- **SIEM Implementation:** Full ELK Stack deployment (Elasticsearch, Logstash, Kibana) with Fleet Server and Elastic Agent for centralized log ingestion from Windows Event Logs, Sysmon, Linux Audited, and network sources.
- **Endpoint Detection & Response (EDR):** Elastic Defend integration for kernel-level endpoint visibility, process monitoring, file integrity, and behavioral detection across all managed endpoints.
- **Identity & Active Directory Security:** Monitoring of authentication events, privilege escalation attempts, Kerberoasting indicators, and Group Policy changes via Windows Security Event Logs forwarded from the Domain Controller.
- **Threat Detection Engineering:** Development of custom detection rules and KQL queries mapped to the MITRE ATT&CK framework to identify adversary techniques including brute force, lateral movement, persistence, and data exfiltration.

1.3 Core Objectives

The project is structured around five strategic objectives that guide every technical and operational decision made during the build and operation of this Mini SOC:

1. **Centralized Visibility:** Aggregate security telemetry from Windows, Linux, and network sources into a single pane of glass, eliminating blind spots across the monitored environment.
2. **Threat Detection & Alerting:** Build and tune detection rules covering the most critical MITRE ATT&CK tactics — from Initial Access and Execution through Persistence, Lateral Movement, and Exfiltration — with measurable low false-positive rates.
3. **Incident Triage Capability:** Establish a structured Tier 1 and Tier 2 triage workflow that enables the analyst team to investigate, classify, and escalate security incidents in alignment with SOC SLA requirements.
4. **Real-World Lab Simulation:** Simulate real attack scenarios (brute force, privilege escalation, lateral movement) within a controlled lab to validate detection coverage and refine response procedures without affecting production systems.

1.4 Technology Stack

The Mini SOC is built on a carefully selected stack of industry-standard and open-source tools that collectively deliver end-to-end security monitoring capabilities:

- **SIEM & Log Management:** ELK Stack 8.x (Elasticsearch, Logstash, Kibana) deployed via Docker on Contabo VPS, providing log storage, processing pipelines, and interactive dashboards.
- **Agent Management & EDR:** Elastic Agent and Elastic Defend (EDR) managed via Fleet Server, deployed on all Windows and Linux endpoints for telemetry collection and behavioral prevention.

- **Identity Infrastructure:** Windows Server 2022 Active Directory Domain Controller (soc.local domain) with Windows 11 Pro domain-joined workstation as a managed endpoint.
- **Detection Framework:** MITRE ATT&CK framework used as the primary reference for detection rule development and threat modeling across all monitored attack surfaces.

1.5 Expected Outcomes

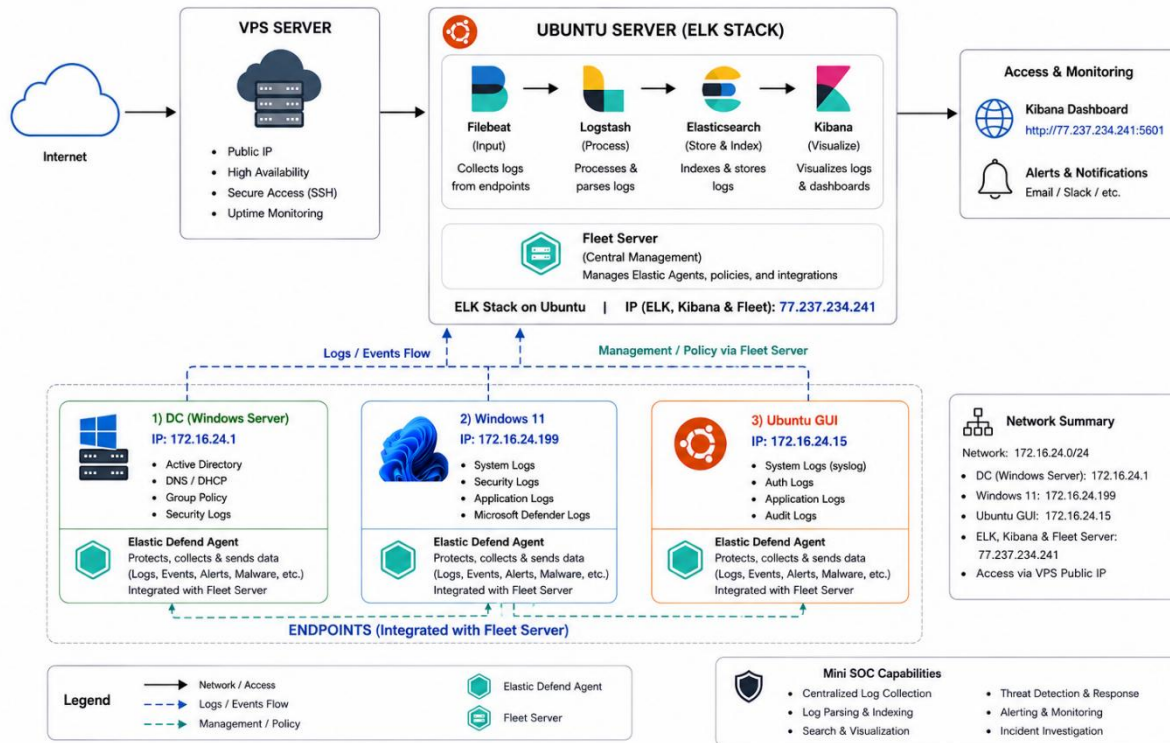
Upon completion of this project, the team will have delivered the following measurable outcomes:

- A fully operational Mini SOC environment with 24/7 log ingestion from at least four distinct data sources (Windows DC, Windows workstation, Ubuntu endpoint, and network telemetry).
- A library of validated detection rules covering the top MITRE ATT&CK techniques relevant to the monitored environment, with documented false-positive tuning notes.
- Custom Kibana dashboards provide real-time security event visibility, alert status tracking, endpoint health monitoring, and authentication anomaly analysis.
- Documented incident response playbooks aligned to the most common alert types generated by the system, enabling consistent and repeatable analyst workflows.
- A comprehensive technical project report demonstrating each team member's contribution, architectural decisions, challenges encountered, and lessons learned throughout the SOC build process.

2. System Architecture & Lab Design

The infrastructure is distributed across three primary environments to ensure high availability and realistic network segmentation.

Mini SOC System – Flow Architecture



A. Contabo VPS — Cloud SOC Platform

- SIEM Stack: Dockerized deployment of ELK Stack (Elasticsearch, Logstash, Kibana).
- Central Management: Fleet Server for managing security policies across all agents which integrated with elastic defend.

B. Adham's Laptop — Infrastructure Cluster

- Identity Management: Windows Server 2022 acting as the Domain Controller (DC).
- Workstation: Windows 11 Pro instance joined to the soc.local domain and Ubuntu GUI Machine to get Linux logs
- Security Layers: Both machines protected by Elastic Agent and Elastic Defend (EDR) for kernel-level visibility.

4. Backend Infrastructure: Elasticsearch & Kibana Deployment

The core of our SOC resides on a Contabo VPS (Ubuntu 22.04) to ensure 24/7 log availability. Elasticsearch 8.x is deployed as the primary search and analytics engine.

Phase 1: Installing Elasticsearch

To ensure stable and secure deployment, follow these commands in order. This process adds the official Elastic repository and installs the latest 8.x version.

Step 1 — Add the GPG Key

```
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg
```

Step 2 — Add the Official Repository

```
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main" | sudo tee /etc/apt/sources.list.d/elasticsearch-8.x.list
```

Step 3 — Update and Install

```
sudo apt update && sudo apt install elasticsearch -y
```

```
root@fqdn:~/soc-home-lab# wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg
root@fqdn:~/soc-home-lab# echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main" | sudo tee /etc/apt/sources.list.d/elasticsearch-8.x.list
deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main
root@fqdn:~/soc-home-lab# sudo apt update && sudo apt install elasticsearch -y
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:3 https://download.docker.com/linux/ubuntu jammy InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Get:5 https://artifacts.elastic.co/packages/8.x/apt stable InRelease [3249 B]
Hit:6 https://download.virtualmin.com/virtualmin InRelease
Hit:7 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Get:8 https://artifacts.elastic.co/packages/8.x/apt stable/main amd64 Packages [185 kB]
Fetched 108 kB in 3s (34.1 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
5 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Phase 2: Performance Optimization (JVM Tuning)

Since we are working with a 12GB RAM environment, we must limit the memory consumption of Elasticsearch to avoid system freezing. We allocate 4GB to the service.

Step 1 — Create Memory Configuration File

```
sudo nano /etc/elasticsearch/jvm.options.d/memory.options
```

Step 2 — Set Heap Size

Inside the file, paste the following lines (ensure there are no leading spaces):

```
-Xms4g
-Xmx4g
```

Press **Ctrl+O** then **Enter** to save, and **Ctrl+X** to exit.

Phase 3: Service Activation & Security

Step 1 — Enable and Start the Service

```
sudo systemctl enable elasticsearch --now
```

Step 2 — Verify Service Status

```
sudo systemctl status elasticsearch
```

```
root@fqdn:~/soc-home-lab# sudo nano /etc/elasticsearch/jvm.options
root@fqdn:~/soc-home-lab# sudo systemctl enable elasticsearch --now
Created symlink /etc/systemd/system/multi-user.target.wants/elasticsearch.service → /lib/systemd/system/elasticsearch.service.
root@fqdn:~/soc-home-lab# sudo systemctl status elasticsearch
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/lib/systemd/system/elasticsearch.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-05-09 00:50:05 CEST; 34s ago
     Docs: https://www.elastic.co
   Main PID: 394899 (java)
    Tasks: 116 (limit: 14298)
   Memory: 4.5G
      CPU: 4min 56.224s
   CGroup: /system.slice/elasticsearch.service
           └─394899 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=server -Dcli.script=/usr/share/
           └─395425 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.networkaddress.cache.negative.ttl=
           └─395511 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/controller

May 09 00:48:37 fqdn.example.com systemd[1]: Starting Elasticsearch...
May 09 00:50:05 fqdn.example.com systemd[1]: Started Elasticsearch.
```

Step 3 — Reset Admin Password

By default, security is enabled in version 8.x. You must generate a new password for the elastic superuser:

```
sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic
```

Important: Copy the generated password immediately and save it in a secure location.

Step 4 — Connection Test

Verify that the API is responding correctly:

```
curl -u elastic -k https://localhost:9200
```

If successful, you will see a JSON response containing the cluster name and version number.

5. Installing and Configuring Kibana

Since the Elastic repositories were added in the previous phase, we can proceed directly to the installation and integration of Kibana as the visualization interface for our SOC.

Step 1 — Install Kibana

```
sudo apt install kibana -y
```

```
root@fqdn:/soc-home-lab# sudo apt install kibana -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  kibana
0 upgraded, 1 newly installed, 0 to remove and 5 not upgraded.
Need to get 394 MB of archives.
After this operation, 1221 MB of additional disk space will be used.
Get:1 https://artifacts.elastic.co/packages/8.x/apt stable/main amd64 kibana amd64 8.19.15 [394 MB]
Ign:1 https://artifacts.elastic.co/packages/8.x/apt stable/main amd64 kibana amd64 8.19.15
Get:1 https://artifacts.elastic.co/packages/8.x/apt stable/main amd64 kibana amd64 8.19.15 [394 MB]
Fetched 394 MB in 49s (8116 kB/s)
[master 809cbbd] saving uncommitted changes in /etc prior to apt run
 4 files changed, 6 insertions(+), 2 deletions(-)
  create mode 100644 elasticsearch/jvm.options.d/memory.options
  create mode 120000 systemd/system/multi-user.target.wants/elasticsearch.service
Selecting previously unselected package kibana.
(Reading database ... 162347 files and directories currently installed.)
Preparing to unpack .../kibana_8.19.15_amd64.deb ...
Unpacking kibana (8.19.15) ...
Setting up kibana (8.19.15) ...
Creating kibana group... OK
Creating kibana user... OK
Kibana is currently running with legacy OpenSSL providers enabled! For details and instructions on how to disable see https://www.elastic.co/guide/en/kibana/8.19/production.html#openssl-legacy-provider
Created Kibana keystore in /etc/kibana/kibana.keystore
[master 0933327] committing changes in /etc made by "apt install kibana -y"
13 files changed, 224 insertions(+)
  create mode 100644 default/kibana
  create mode 100644 kibana/kibana.keystore
```

Step 2 — Generate Enrollment Token

To securely link Kibana with Elasticsearch, generate a one-time Enrollment Token:

```
sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana
```

Action: Copy the long alphanumeric string generated by this command. You will need it for the initial web configuration.

[illegible]

Step 3 — Enable and Start the Kibana Service

```
sudo systemctl enable kibana --now
```

```
root@fqdn:~/soc-home-lab# sudo systemctl enable kibana --now
Created symlink /etc/systemd/system/multi-user.target.wants/kibana.service → /lib/systemd/system/kibana.service.
root@fqdn:~/soc-home-lab#
```

Step 4 — Verify Service Status

```
sudo systemctl status kibana
```


Step 5 — Access the Web Interface

Open your browser and navigate to your VPS IP address on port 5601:

```
https://77.237.234.241:5601
```

6. SSL/TLS Certificate Generation

To ensure secure, encrypted communication between Elastic Agents and the Fleet Server, we use the `elasticsearch-certutil` tool to generate self-signed certificates.

Step 1 — Generate the Certificate Authority (CA)

Create a Certificate Authority to sign certificates for our server nodes:

```
sudo /usr/share/elasticsearch/bin/elasticsearch-certutil ca
```

```
root@fqdn:~/soc-home-lab# sudo /usr/share/elasticsearch/bin/elasticsearch-certutil ca --pem
This tool assists you in the generation of X.509 certificates and certificate
signing requests for use with SSL/TLS in the Elastic stack.
```

```
The 'ca' mode generates a new 'certificate authority'
This will create a new X.509 certificate and private key that can be used
to sign certificate when running in 'cert' mode.
```

```
Use the 'ca-dn' option if you wish to configure the 'distinguished name'
of the certificate authority
```

```
By default the 'ca' mode produces a single PKCS#12 output file which holds:
* The CA certificate
* The CA's private key
```

```
If you elect to generate PEM format certificates (the -pem option), then the output will
be a zip file containing individual files for the CA certificate and private key
```

```
root@fqdn:~/soc-home-lab# sudo unzip elastic-stack-ca.zip
unzip: cannot find or open elastic-stack-ca.zip, elastic-stack-ca.zip.zip or elastic-stack-ca.zip.ZIP.
root@fqdn:~/soc-home-lab# sudo /usr/share/elasticsearch/bin/elasticsearch-certutil cert --name fleet-server --ca-cert ca/ca.crt --ca-
key ca/ca.key --pem --dns 77.237.234.241 --ip 77.237.234.241
This tool assists you in the generation of X.509 certificates and certificate
signing requests for use with SSL/TLS in the Elastic stack.
```

```
The 'cert' mode generates X.509 certificate and private keys.
* By default, this generates a single certificate and key for use
  on a single instance.
* The '-multiple' option will prompt you to enter details for multiple
  instances and will generate a certificate and key for each one
* The '-in' option allows for the certificate generation to be automated by describing
  the details of each instance in a YAML file

* An instance is any piece of the Elastic Stack that requires an SSL certificate.
  Depending on your configuration, Elasticsearch, Logstash, Kibana, and Beats
  may all require a certificate and private key.
* The minimum required value for each instance is a name. This can simply be the
  hostname, which will be used as the Common Name of the certificate. A full
  distinguished name may also be used.
* A filename value may be required for each instance. This is necessary when the
  name would result in an invalid file or directory name. The name provided here
  is used as the directory name (within the zip) and the prefix for the key and
  certificate files. The filename is required if you are prompted and the name
  is not displayed in the prompt.
```

```
root@fqdn:~/soc-home-lab# sudo mkdir -p /etc/kibana/certs
root@fqdn:~/soc-home-lab# sudo cp fleet-server.crt fleet-server.key ca/ca.crt /etc/kibana/certs/
cp: cannot stat 'fleet-server.crt': No such file or directory
cp: cannot stat 'fleet-server.key': No such file or directory
cp: cannot stat 'ca/ca.crt': No such file or directory
root@fqdn:~/soc-home-lab#
```

7. Troubleshooting & Final Enrollment

Issue: Access Denied Exception

During the deployment phase, the Elasticsearch service failed to initialize due to a critical "Access Denied" error. The service could not read the required security tokens.

Root Cause

Incorrect file permissions and ownership on the service tokens file.

Resolution

- Ownership Adjustment: Transferred ownership of the service tokens file to the elasticsearch user and group.
- Permission Hardening: Set file permissions to 660 — only the service and authorized group can read or write.

Correction Commands

```
# Correcting ownership of the service tokens
sudo chown elasticsearch:elasticsearch /etc/elasticsearch/service_tokens

# Setting secure permissions (Read/Write for Owner & Group)
sudo chmod 660 /etc/elasticsearch/service_tokens
```

Final Fleet Server Enrollment

After confirming that the Elasticsearch service was Active (running) and verifying sufficient system resources (approximately 9.9GB of available memory), we proceeded with the final enrollment of the Fleet Server. To accommodate potential network latency, the operation timeout was increased to 10 minutes.

Final Enrollment Command

```
sudo elastic-agent enroll \
  --url=https://77.237.234.241:8220 \
  --fleet-server-es=https://localhost:9200 \
  --fleet-server-es-ca-trusted-fingerprint=[YOUR_FINGERPRINT] \
  --fleet-server-service-token=[YOUR_TOKEN] \
  --fleet-server-policy=fleet-server-policy \
  --fleet-server-es-insecure \
  --insecure \
  --fleet-server-timeout=10m
```

Add a Fleet Server

A Fleet Server is required before you can enroll agents with Fleet. Follow the instructions below to set up a Fleet Server. For more information, see the [Fleet and Elastic Agent Guide](#).

Quick Start

Advanced

✓

Get started with Fleet Server

✓ Fleet Server policy created.

Fleet server policy and service token have been generated. Host configured at <https://77.237.234.241:8220>. You can edit your Fleet Server hosts in [Fleet Settings](#).

2

Install Fleet Server to a centralized host

Install Fleet Server agent on a centralized host so that other hosts you wish to monitor can connect to it. In production, we recommend using one or more dedicated hosts. For additional guidance, see our [installation docs](#).

Linux Tar

Mac

Windows

RPM

DEB

```
root@fqdn:~/soc-home-lab# curl -L -O https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-8.11.0-linux-x86_64.tar.gz
tar xzvf elastic-agent-8.11.0-linux-x86_64.tar.gz
cd elastic-agent-8.11.0-linux-x86_64
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 545M 100 545M 0 0 5752k 0 0:01:37 0:01:37 --:--:-- 6181k
elastic-agent-8.11.0-linux-x86_64/.build_hash.txt
elastic-agent-8.11.0-linux-x86_64/NOTICE.txt
elastic-agent-8.11.0-linux-x86_64/README.md
elastic-agent-8.11.0-linux-x86_64/data/elastic-agent-3c8be7/components/
elastic-agent-8.11.0-linux-x86_64/data/elastic-agent-3c8be7/components/.build_hash.txt
```

```
root@fqdn:~# sudo elastic-agent enroll \
--url=https://77.237.234.241:8220 \
--fleet-server-es=https://localhost:9200 \
--fleet-server-es-ca-trusted-fingerprint=B3AF7D86DB7224768994CDD1FFE0A5627ADD09DA72595AAED9FA0ED4FB2A487 \
--fleet-server-service-token=AAEAaWVsYXN0aWVzMmxLZXQtc2VydmlvL3Rva2VuLTE3NzgyODgyMjc0MTM6VXZfB0tta0dRTXk2c0hRdXhNNW1qZW \
--fleet-server-policy=fleet-server-policy \
--fleet-server-es-insecure \
--insecure \
--fleet-server-timeout=15m
This will replace your current settings. Do you want to continue? [Y/n]:y
{"log.level":"info","@timestamp":"2026-05-09T03:42:53.746+0200","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/cmd.(*enrollCmd).prepareFleetTLS","file.name":"cmd/enroll_cmd.go","file.line":464},"message":"Generating self-signed certificate for Fleet Server","ecs.version":"1.6.0"}
{"log.level":"info","@timestamp":"2026-05-09T03:42:54.302+0200","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/cmd.(*enrollCmd).daemonReloadWithBackoff","file.name":"cmd/enroll_cmd.go","file.line":508},"message":"Restarting agent daemon, attempt 0","ecs.version":"1.6.0"}
{"log.level":"info","@timestamp":"2026-05-09T03:42:56.314+0200","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/cmd.waitForFleetServer.func1","file.name":"cmd/enroll_cmd.go","file.line":862},"message":"Fleet Server - Starting","ecs.version":"1.6.0"}
{"log.level":"info","@timestamp":"2026-05-09T03:43:00.335+0200","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/cmd.waitForFleetServer.func1","file.name":"cmd/enroll_cmd.go","file.line":862},"message":"Fleet Server - Starting","ecs.version":"1.6.0"}
```

8. Install Elastic Defend Agent on Endpoints

Step 1: Create Elastic Defend Policy

5. Go to **Kibana**
6. Navigate to:
 - Security → Manage → Policies
7. Click **Create Policy**
8. Enable:
 - Elastic Defend Integration
9. Select protection mode:
 - Prevent / Detect (based on lab setup)

View all agent policies

Revision 7 Integrations 4 Agents 4 agents Last updated on May 12, 2026 Actions

Windows-Linux-Defend

Integrations Settings

Search... Namespace Add integration

Integration policy ↑	Integration ↓	Namespace	Output	Actions
network_traffic-2	Network Packet Capture v1.34.2	default	default	...
system-3	System v2.6.3	default	default	...
windows-1	Windows v3.8.3	default	default	...
winlog-1	Custom Windows Event Logs v2.6.0	default	default	...

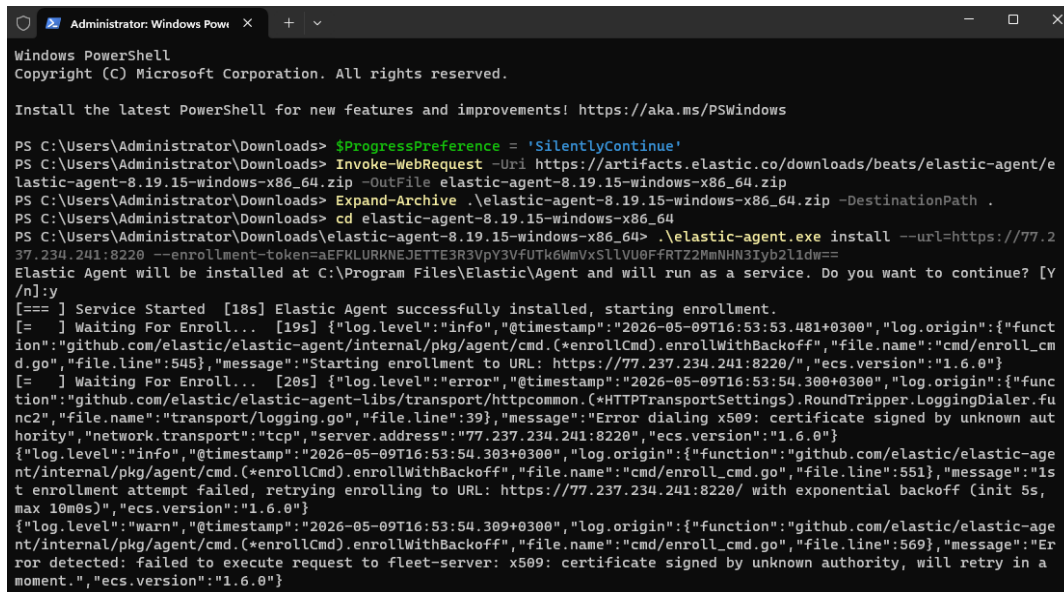
Step 2: Add Agent in Fleet

1. Go to:
 - Management → Fleet → Agents
2. Click **Add Agent**
3. Select the created policy (Elastic Defend Policy)
4. Choose platform:
 - Windows / Linux / Ubuntu
5. Copy the installation command provided by Kibana

☐	Offline	DESKTOP-5657BDI	Windows-Linux-Defend rev. 7	N/A	N/A	17 hours ago	8.19.15	...
☐	Healthy	fqdn.example.com F-S Ba3d el-In3ash	Fleet Server Policy rev. 7	5.54 %	551 MB	37 seconds ago	8.19.15	...
☐	Offline	fqdn.example.com	VPS-Mail-Policy	N/A	N/A		8.19.15	...
☐	Offline	SOC-PC01	Windows-Linux-Defend rev. 7	N/A	N/A	1 hour ago	8.19.15	...
☐	Offline	WIN-ASKH2JHD0DP	Windows-Linux-Defend rev. 7	N/A	N/A	4 hours ago	8.19.15	...
☐	Offline	SOCLAB	Windows-Linux-Defend rev. 7	N/A	N/A	1 hour ago	8.19.15	...

Step 3: Install Elastic Defend Agent on Endpoints (Windows – Linux)

```
$ProgressPreference = 'SilentlyContinue'
Invoke-WebRequest -Uri https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-8.19.15-windows-x86_64.zip -OutFile elastic-agent-8.19.15-windows-x86_64.zip
Expand-Archive .\elastic-agent-8.19.15-windows-x86_64.zip -DestinationPath .
cd elastic-agent-8.19.15-windows-x86_64
.\elastic-agent.exe install --url=https://77.237.234.241:8220 --enrollment-token=aEFKLURKNEJETTE3R3VpY3VfUtk6WmVxS1lVU0FfRTZ2MmNHN3Iyb2l1dw==
```

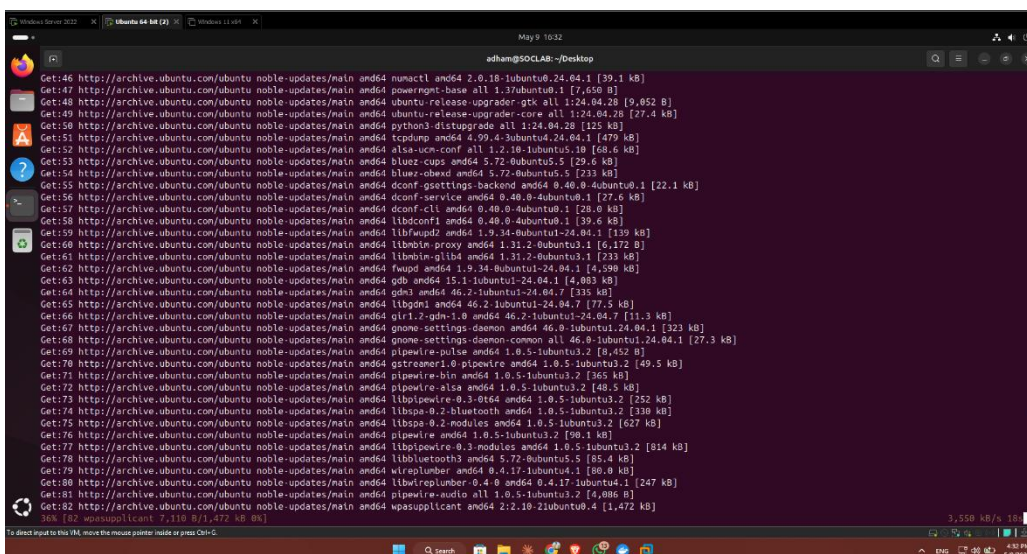


```
Administrator: Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Administrator\Downloads> $ProgressPreference = 'SilentlyContinue'
PS C:\Users\Administrator\Downloads> Invoke-WebRequest -Uri https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-8.19.15-windows-x86_64.zip -OutFile elastic-agent-8.19.15-windows-x86_64.zip
PS C:\Users\Administrator\Downloads> Expand-Archive .\elastic-agent-8.19.15-windows-x86_64.zip -DestinationPath .
PS C:\Users\Administrator\Downloads> cd elastic-agent-8.19.15-windows-x86_64
PS C:\Users\Administrator\Downloads> .\elastic-agent.exe install --url=https://77.237.234.241:8220 --enrollment-token=aEFKLURKNEJETTE3R3VpY3VfUtk6WmVxS1lVU0FfRTZ2MmNHN3Iyb2l1dw==
Elastic Agent will be installed at C:\Program Files\Elastic\Agent and will run as a service. Do you want to continue? [Y/n]:y
[=== ] Service Started [18s] Elastic Agent successfully installed, starting enrollment.
[= ] Waiting For Enrollment... [19s] {"log.level":"info","@timestamp":"2026-05-09T16:53:53.481+0300","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/cmd.(*enrollCmd).enrollWithBackoff","file.name":"cmd/enroll_cmd.go","file.line":545},"message":"Starting enrollment to URL: https://77.237.234.241:8220/", "ecs.version":"1.6.0"}
[= ] Waiting For Enrollment... [20s] {"log.level":"error","@timestamp":"2026-05-09T16:53:54.300+0300","log.origin":{"function":"github.com/elastic/elastic-agent-libs/transport/httpcommon.(*HTTPTransportSettings).RoundTripper.LoggingDialer.func2","file.name":"transport/logging.go","file.line":39},"message":"Error dialing x509: certificate signed by unknown authority","network.transport":"tcp","server.address":"77.237.234.241:8220","ecs.version":"1.6.0"}
{"log.level":"info","@timestamp":"2026-05-09T16:53:54.303+0300","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/cmd.(*enrollCmd).enrollWithBackoff","file.name":"cmd/enroll_cmd.go","file.line":551},"message":"1st enrollment attempt failed, retrying enrolling to URL: https://77.237.234.241:8220/ with exponential backoff (init 5s, max 10m0s)","ecs.version":"1.6.0"}
{"log.level":"warn","@timestamp":"2026-05-09T16:53:54.309+0300","log.origin":{"function":"github.com/elastic/elastic-agent/internal/pkg/agent/cmd.(*enrollCmd).enrollWithBackoff","file.name":"cmd/enroll_cmd.go","file.line":569},"message":"Error detected: failed to execute request to fleet-server: x509: certificate signed by unknown authority, will retry in a moment.","ecs.version":"1.6.0"}
```

```
curl -L -O https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-8.19.15-linux-x86_64.tar.gz
tar xzvf elastic-agent-8.19.15-linux-x86_64.tar.gz
cd elastic-agent-8.19.15-linux-x86_64
sudo ./elastic-agent install --url=https://77.237.234.241:8220 --enrollment-token=aEFKLURKNEJETTE3R3VpY3VfUtk6WmVxS1lVU0FfRTZ2MmNHN3Iyb2l1dw==
```



```
adham@SOCLAB:~/Desktop
Get:46 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 numactl amd64 2.0.18-1ubuntu0.24.04.1 [39.1 kB]
Get:47 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 powermgmt-base all 1.37ubuntu0.1 [7,650 B]
Get:48 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 ubuntu-release-upgrader-gtk all 1:24.04.28 [9,652 B]
Get:49 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 ubuntu-release-upgrader-core all 1:24.04.28 [27.4 kB]
Get:50 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 python3-distupgrader all 1:24.04.28 [125 kB]
Get:51 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 tcpdump amd64 4.99.4-3ubuntu0.24.04.1 [479 kB]
Get:52 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 elsa-ucm-conf all 1:2.18-1ubuntu0.1 [68.6 kB]
Get:53 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 bluez-cups amd64 5.72-0ubuntu0.5 [29.6 kB]
Get:54 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 bluez-obsd amd64 5.72-0ubuntu0.5 [233 kB]
Get:55 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 dconf-gsettings-backend amd64 0.40.0-4ubuntu0.1 [22.1 kB]
Get:56 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 dconf-service amd64 0.40.0-4ubuntu0.1 [27.6 kB]
Get:57 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 dconf-clients amd64 0.40.0-4ubuntu0.1 [20.0 kB]
Get:58 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libdconf1 amd64 0.40.0-4ubuntu0.1 [39.6 kB]
Get:59 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libfwupd2 amd64 1.9.34-0ubuntu1-24.04.1 [139 kB]
Get:60 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libbluetooth-proxy amd64 1.31.2-0ubuntu0.1 [6,172 B]
Get:61 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libbluetooth-glib amd64 1.31.2-0ubuntu0.1 [233 kB]
Get:62 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 fwupd amd64 1.9.34-0ubuntu1-24.04.1 [4,598 kB]
Get:63 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 gdb amd64 15.1-1ubuntu1-24.04.1 [4,083 kB]
Get:64 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 glibc amd64 46.2-1ubuntu1-24.04.7 [335 kB]
Get:65 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libglib2.0-0 amd64 46.2-1ubuntu1-24.04.7 [77.5 kB]
Get:66 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 gir1.2-glib-1.0 amd64 46.2-1ubuntu1-24.04.7 [11.3 kB]
Get:67 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 gnomine-daemon amd64 46.0-1ubuntu1-24.04.1 [323 kB]
Get:68 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 gnomine-settings-daemon-common all 46.0-1ubuntu1-24.04.1 [27.3 kB]
Get:69 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 pipewire-pulse amd64 1.0.5-1ubuntu0.2 [9,452 B]
Get:70 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 gstreame1.0-pipewire amd64 1.0.5-1ubuntu0.2 [49.5 kB]
Get:71 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 pipewire-bin amd64 1.0.5-1ubuntu0.2 [365 kB]
Get:72 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 pipewire-alsa amd64 1.0.5-1ubuntu0.2 [48.5 kB]
Get:73 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libpipewire-0.3-0 amd64 1.0.5-1ubuntu0.2 [252 kB]
Get:74 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libspa-0.2-bluetooth amd64 1.0.5-1ubuntu0.2 [330 kB]
Get:75 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libspa-0.2-modules amd64 1.0.5-1ubuntu0.2 [627 kB]
Get:76 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 pipewire amd64 1.0.5-1ubuntu0.2 [390.1 kB]
Get:77 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libpipewire-0.3-modules amd64 1.0.5-1ubuntu0.2 [814 kB]
Get:78 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libbluetooth3 amd64 5.72-0ubuntu0.5 [85.4 kB]
Get:79 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 wireplumber amd64 0.4.17-1ubuntu0.1 [80.8 kB]
Get:80 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libwireplumber-0.4-0 amd64 0.4.17-1ubuntu0.1 [247 kB]
Get:81 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 pipewire-audio all 1.0.5-1ubuntu0.2 [5,005 B]
Get:82 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 wpasupplicant amd64 2:2.10-21ubuntu0.4 [1,472 kB]
368 [82 wpasupplicant 7,110 B/1,472 kB OK]
3,550 kB/s 10
To direct input to this VM, move the mouse pointer inside or press Ctrl-Q.
```

Step 4: Verify Healthy Enrollment

Fleet

Centralized management for Elastic Agents.

[Agents](#)
[Agent policies](#)
[Enrollment tokens](#)
[Uninstall tokens](#)
[Data streams](#)
[Settings](#)

[Ingest Overview Metrics](#)
[Agent Info Metrics](#)
[Agent activity](#)
[Add Fleet Server](#)
[Add agent](#)

Filter your data using KQL syntax

Status 4
 Tags 1
 Agent policy 3
 Upgrade available

Showing 4 agents [Clear filters](#)

Healthy 4
 Unhealthy 0
 Updating 0
 Offline 0
 Inactive 0
 Unenrolled 0

☐	Status	Host	Agent policy	CPU	Memory	Last activity	Version	Actions
☐	Healthy	SOC-PC01	Windows-Linux-Defend rev. 4	0.83 %	300 MB	33 seconds ago	8.19.15	...
☐	Healthy	WIN-ASKH2JHD0DP	Windows-Linux-Defend rev. 4	1.51 %	301 MB	17 seconds ago	8.19.15	...
☐	Healthy	SOCCLAB	Windows-Linux-Defend rev. 4	1.41 %	253 MB	17 seconds ago	8.19.15	...
☐	Healthy	fqdn.example.com Lab-Main-Server	Fleet Server Policy rev. 5	5.15 %	357 MB	37 seconds ago	8.19.15	...

Rows per page: 20

9.Create Custom rules

As part of the Mini SOC implementation, several custom detection rules were created in Kibana to identify common attack patterns and simulate real-world threat scenarios.

- Windows Failed Login (Event ID 4625):**
 Detects authentication failures to identify unauthorized access attempts and possible brute-force attacks.
- SSH Brute Force (Linux):**
 Monitors repeated failed SSH login attempts to detect brute-force activity targeting Linux systems.
- SQL Injection Attempts:**
 Identifies suspicious web requests containing malicious SQL patterns such as UNION, SELECT, and DROP

Security

Dashboards
 Rules
 Alerts
 Attack discovery
 Findings
 Cases
 Timelines
 Intelligence
 Explore

Detection rules (SIEM)

[Add Elastic rules](#)
[Manage value lists](#)
[Import rules](#)
[Create new rule](#)

Installed Rules 1547
 Rule Monitoring 1647

Rule name, index pattern (e.g., "filebeat-*"), or MITRE ATT&CK tactic or technique (e.g., "Defense Evasion")
 Tags 213
 Last response 3
 Elastic rules (1644)
 Custom rules (3)
 Enabled rules
 Disabled rules

Showing 1-3 of 3 rules

☐	Rule	Risk	Severity	Last run	Last response	Last updated	Notify	Enabled	Actions
☐	SQL Injection Attempt on OWASP Juice Shop	73	High	2 minutes ago	Succeeded	May 18, 2026 @ 06:26...		☒	...
☐	Windows - Failed Login	73	High	1 minute ago	Succeeded	1 minute ago		☒	...
☐	SSH Failed Login on VPS	73	High	10 seconds ago	Succeeded	yesterday		☒	...

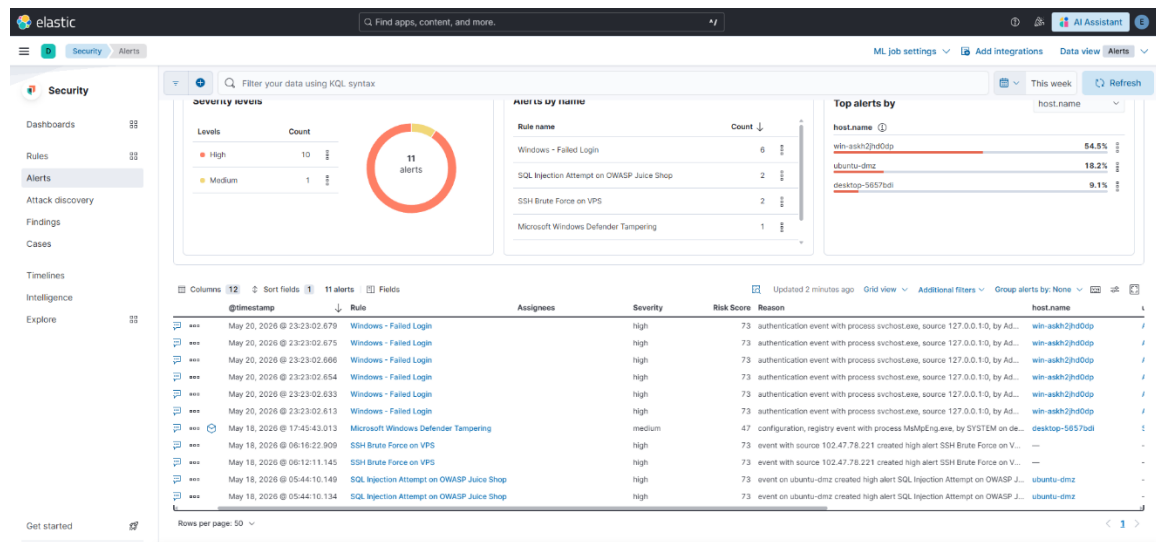
Rows per page: 20

10. Alerts & Incident Notification

The Mini SOC system generates real-time alerts based on the defined detection rules in Kibana. These alerts provide immediate visibility into suspicious activities such as failed logins, brute-force attempts, and potential web attacks.

Each alert includes key details such as the affected host, user account, source IP, and event context, enabling efficient triage and investigation. Alerts are prioritized based on severity levels (Low, Medium, High) to help analysts focus on critical threats.

This alerting mechanism ensures rapid detection, streamlined incident response, and improved overall security monitoring within the SOC environment.



11. Conclusion

• Infrastructure Summary & Lessons Learned

The Integrated Mini-SOC Ecosystem successfully demonstrated the design, deployment, and operationalization of a production-grade security monitoring environment built entirely on open-source and cloud-based technologies. From an infrastructure standpoint, the project validated a hybrid architecture model — combining cloud VPS hosting with on-premises virtualized endpoints — as a cost-effective and scalable foundation for a functional SOC.

• Architecture Validation

The decision to host the SIEM core (Elasticsearch 8.x + Kibana + Fleet Server) on a Contabo VPS running Ubuntu 22.04 proved to be the right call for ensuring 24/7 log availability and centralized policy management. Decoupling the data collection layer (endpoints) from the analytics layer (cloud)

introduced realistic network segmentation and accurately mirrors enterprise-grade SOC architectures where agents report to a centralized platform over the internet.

The on-premises cluster — consisting of a Windows Server 2022 Domain Controller, a Windows 11 Pro domain-joined workstation, and an Ubuntu GUI machine — provided a diverse, multi-OS log source environment. This setup generated meaningful telemetry covering Windows Event Logs, Active Directory authentication events, and Linux syslog data, all forwarded to the ELK stack via Elastic Agent with Elastic Defend for kernel-level EDR visibility.

- **Deployment Challenges & Engineering Decisions**

The project encountered real-world infrastructure challenges that added significant learning value. The JVM heap tuning (setting `-Xms4g / -Xmx4g` on a 12GB RAM VPS) was a critical optimization step to prevent Elasticsearch from consuming all available memory and causing service instability. This is a common pain point in production Elastic deployments and was handled correctly.

The Access Denied exception during Fleet Server enrollment — caused by incorrect file ownership on `/etc/elasticsearch/service_tokens` — is a classic Linux permission hardening issue. Resolving it through `chown` and `chmod 660` reflects solid sysadmin fundamentals applied in a security context.

SSL/TLS certificate generation using `elasticsearch-certutil` for securing agent-to-Fleet communication further strengthened the infrastructure's security posture and ensured encrypted log transport across all nodes.

- **Key Outcomes**

The final infrastructure achieved the following operational capabilities:

- Centralized ingestion and indexing of multi-source security logs in near real-time
- Agent-based endpoint coverage across Windows and Linux platforms via Elastic Defend (EDR)
- Fleet-managed policy enforcement across all enrolled agents from a single control plane
- Secure communications enforced via self-signed TLS certificates throughout the stack

- **Final Assessment**

This project delivered a technically sound, end-to-end SOC infrastructure that bridges the gap between theoretical SOC architecture and hands-on deployment. Every component — from OS-level configuration to certificate management and agent enrollment — was implemented with security and operational stability in mind. The experience gained directly mirrors the challenges faced by infrastructure and SOC teams in enterprise environments, making it a strong foundation for real-world blue team operations.